

**INSTITUTO FEDERAL DE EDUCAÇÃO, CIÊNCIA E TECNOLOGIA DE
SÃO PAULO**

NOLAN LEMES MARCONDES

**COWGAME: NOLAN'S FARM – DESENVOLVIMENTO DE
UM JOGO EM 2D UTILIZANDO C++ E ARQUITETURA EN-
CAPSULADA COM A BIBLIOTECA RAYLIB**

CAMPOS DO JORDÃO

2026

RESUMO

O presente trabalho apresenta o desenvolvimento do software de entretenimento intitulado *CowGame: Nolan's Farm*, um jogo eletrônico bidimensional desenvolvido sob o paradigma da Orientação a Objetos na linguagem C++ com o auxílio da biblioteca gráfica Raylib. O objetivo principal deste projeto é aplicar conceitos de engenharia de software, encapsulamento e máquinas de estado no desenvolvimento de uma simulação interativa de fazenda. A metodologia englobou pesquisa bibliográfica e engenharia reversa de jogos retrô, culminando na implementação de sistemas estruturados para IA de NPCs, controle de colisões e persistência temporária de estados de jogo. Como resultado, obteve-se um protótipo funcional estável com renderização em upscaling bilinear, sistema de economia interno, mecânica de tempo dinâmico e inteligência artificial para predadores. Conclui-se que o uso combinado de C++ e Raylib oferece uma plataforma didática robusta para o aprendizado prático de padrões de projeto e gerenciamento de memória em tempo real.

Palavras-Chave: Orientação a Objetos; Jogo 2D; C++; Raylib; Engenharia de Software.

ABSTRACT

This work presents the development of the entertainment software entitled CowGame: Farm Deluxe Edition, a two-dimensional electronic game developed under the Object-Oriented paradigm in the C++ language with the aid of the Raylib graphics library. The main objective of this project is to apply concepts of software engineering, encapsulation, and state machines in the development of an interactive farm simulation. The methodology encompassed bibliographic research and reverse engineering of retro games, culminating in the implementation of structured systems for NPC AI, collision control, and temporary persistence of game states. As a result, we obtained a stable functional prototype with bilinear upscaling rendering, an internal economy system, dynamic time mechanics, and artificial intelligence for predictions. It is concluded that the combined use of C++ and Raylib offers a robust didactic platform for the practical learning of design patterns and real-time memory management.

Keywords: Object-Oriented Programming; 2D Game Design; C++; Raylib; Software Engineering.

LISTA DE ILUSTRAÇÕES

FIGURA 01 – Diagrama UML.	10
FIGURA 02 – Interface inicial do Menu Principal do jogo.	11
FIGURA 03 – Demonstração do jogo estação VERÃO.	12
FIGURA 04 – Demonstração do jogo estação PRIMAVERA.	12
FIGURA 05 – Demonstração do jogo estação OUTONO.	13
FIGURA 06 – Demonstração do jogo estação INVERNO.	13
FIGURA 07 – Demonstração do jogo clima CHUVOSO.	14
FIGURA 08 – Demonstração do jogo entidade LOBO	14
FIGURA 09 – Demonstração do jogo entidade LOBO INVERNO.	15
FIGURA 10 – Demonstração tela de seleção Personagem.	15
FIGURA 11 – Demonstração tela Tutorial.	16
FIGURA 12 – Demonstração tela Game Over.	17

SUMÁRIO

1	INTRODUÇÃO	06
1.1	Objetivos	06
1.2	Justificativa	06
1.3	Aspectos Metodológicos	07
1.4	Aporte Teórico	07
2	Metodologia	09
2.1	Considerações iniciais e ferramentas	09
2.2	Estrutura e Desenvolvimento do jogo	09
2.3	Diagrama UML	10
3	Resultados Obtidos	10
3.1	Tela do menu principal e seleção de modos	10
3.2	Interface da gameplay (HUD e Renderização	11
3.3	Interface de seleção de personagens	15
3.4	Interface de tutorial	16
3.5	Interface de Game over e estatísticas gerais	16
4	CONCLUSÃO	17
	REFERÊNCIAS	19

1 INTRODUÇÃO

Neste capítulo, introduzem-se os aspectos fundamentais que motivaram a concepção de *CowGame: Nolan's Farm*, delimitando os seus objetivos, a justificativa para a escolha das tecnologias, as bases metodológicas e o aporte teórico que sustenta a implementação do software.

1.1 Objetivos

Este trabalho tem por objetivo desenvolver um jogo digital 2D interativo de gerenciamento de fazenda utilizando C++ e Raylib. Para a consecução deste objetivo, estabeleceram-se os seguintes objetivos específicos:

- Implementar uma arquitetura modular baseada em classes (Game, Fazendeiro, Vaca, Lobo, Calendario) com alto nível de encapsulamento;
- Desenvolver um sistema de colisão e física aplicada a climas dinâmicos (inércia em terrenos de gelo e penalidade de velocidade em poças de lama);
- Criar mecânicas de gameplay balanceadas divididas em dois modos distintos de jogo (Diversão e Desafio) para avaliar a adaptabilidade da lógica do motor.

1.2 Justificativa

A relevância do trabalho e a sua justificativa fundamentam-se na necessidade de consolidar o aprendizado prático em Análise e Desenvolvimento de Sistemas através de um projeto de alta complexidade em lógica de programação e gerência de recursos. O desenvolvimento de jogos exige controle rígido de ciclos de execução por segundo (*frames*) e gerenciamento de memória, diferenciando-se de sistemas comerciais CRUD convencionais.

1.3 Aspectos Metodológicos

O presente estudo fez uso de pesquisas de natureza bibliográfica para fundamentar o ciclo de desenvolvimento do motor de jogo e de engenharia prática de software para a construção dos artefatos digitais. O projeto foi dividido em: análise de requisitos, modelagem de classes e loop principal de renderização.

1.4 Aporte Teórico

A pesquisa baseia-se em conceitos estruturais de Computação Gráfica e Arquitetura de Software. O jogo desenvolvido é um simulador de fazenda em perspectiva *Top-Down* (vista superior) onde o jogador controla um fazendeiro (com escolha de gênero do sprite).

- **História:** O protagonista assume o controle de um pequeno rebanho de vacas e precisa gerenciar a alimentação e a ordenha diária para manter o negócio funcionando, lidando com intempéries climáticas e ataques de predadores selvagens.
- **Mecânicas (Gameplay):** O loop principal consiste em coletar ração no silo para alimentar as vacas (evitando que passem fome) e realizar a ordenha utilizando o botão de interação. O leite coletado no balde possui um limite físico de capacidade (ajustável via upgrades); o jogador deve ir até o Tanque Central para descarregar o leite acumulado e validar o progresso.
- **Modos de Jogo:** O modo **Diversão** oferece uma experiência tranquila sem falhas punitivas imediatas. O modo **Desafio** impõe metas diárias crescentes de litros de leite e um temporizador regressivo rígido; caso o tempo acabe antes do cumprimento da meta, o estado é alternado para *Game Over*.

2 METODOLOGIA

2.1 Considerações Iniciais e Ferramentas

O desenvolvimento do projeto seguiu os padrões estabelecidos na especificação de sistemas orientados a objetos. Para a codificação, utilizou-se o ambiente de desenvolvimento integrado (IDE) **Qt Creator**, integrado ao sistema de build **CMake** e utilizando o compilador **MinGW (GCC/G++)** para a linguagem C++ (Padrão ISO C++17). A manipulação de janelas, entradas de teclado e contextos gráficos foi gerenciada através da biblioteca **Raylib v4.5**, vinculada de forma estática ao executável e escolhida por sua API enxuta e alto desempenho para renderização direta via OpenGL no ambiente Windows.

2.2 Estrutura e Desenvolvimento do jogo

O jogo foi estruturado sob o padrão arquitetural clássico de jogos digitais baseado em três fases sequenciais contidas dentro do laço while (!WindowShouldClose()) localizado na classe principal Game::Executar():

- **ProcessarEntradas():** Escuta ativamente as ações do usuário (Teclas W, A, S, D e SETAS para movimentação, SPACE para descarregar leite ou abastecer ração, C para ordenhar e V para alimentar vacas).
- **AtualizarLogica():** Executa as atualizações físicas da aplicação. Aqui, o sistema checa o estado atual do jogo (estadoAtual), atualiza o relógio dinâmico através do objeto calendario, simula o algoritmo de busca do Lobo atrás do rebanho e computa a redução de atrito ou velocidade caso o fazendeiro colida com poças de lama ou com o lago de gelo.
- **RenderizarTela():** Realiza o desenho dos objetos. Para garantir que o aspecto estético em Pixel Art não seja distorcido em telas de monitores diferentes, a aplicação utiliza uma RenderTexture2D com resolução nativa de

800x600 pixels. Tudo é renderizado nessa textura e, ao fim do frame, expandido (*upscaling*) para a resolução da tela do usuário utilizando o filtro bilinear da GPU (TEXTURE_FILTER_BILINEAR).

2.3 Diagrama UML

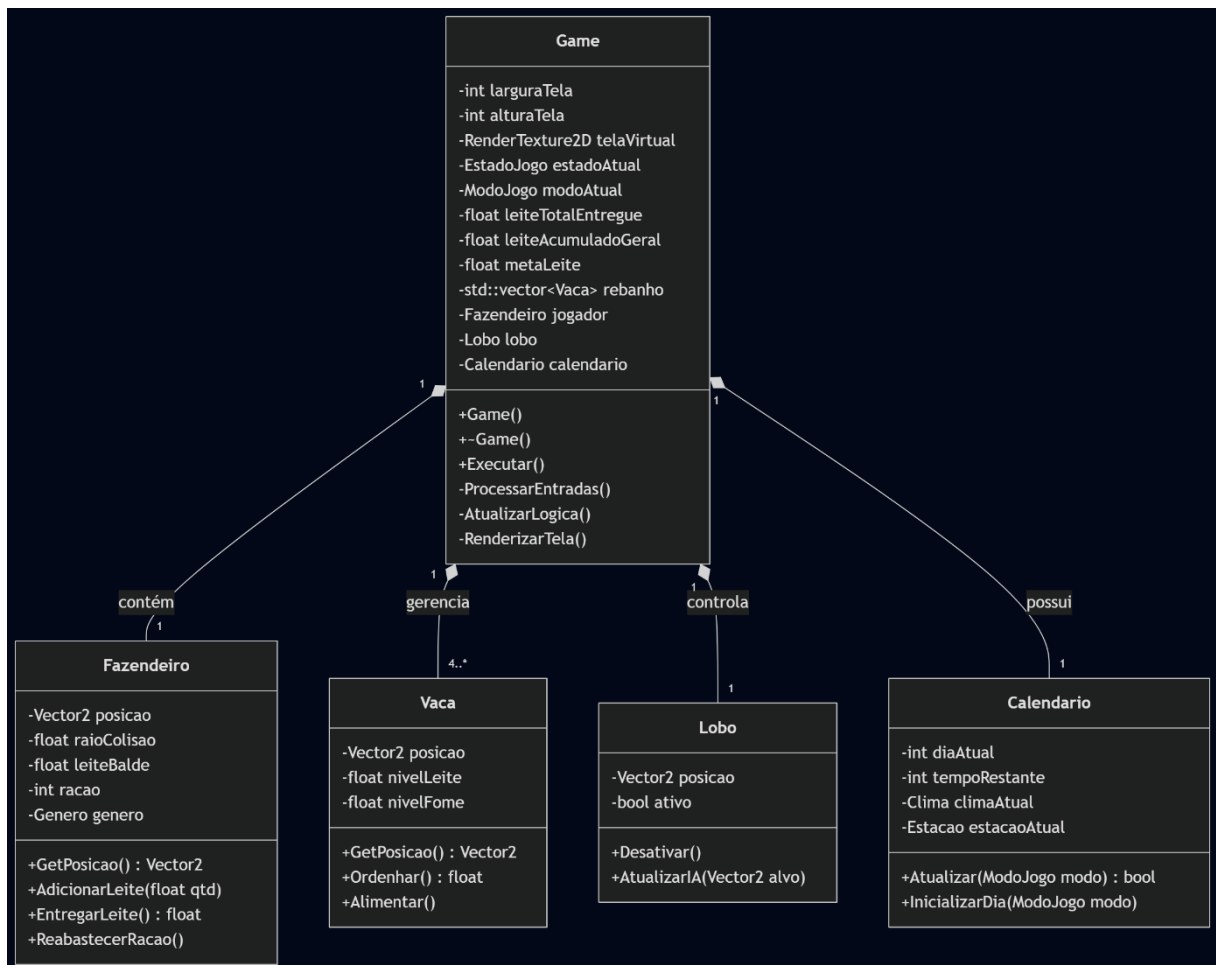


Figura 1 – Diagrama UML

3 RESULTADOS OBTIDOS

Abaixo são documentadas as capturas de tela obtidas diretamente do protótipo final compilado do sistema, demonstrando o comportamento visual das mecânicas descritas:

3.1 Tela do Menu Principal e Seleção de modos



Figura 2 – Interface inicial do Menu Principal do jogo. O usuário escolhe o modo de jogo através das teclas numéricas, o que redefine internamente os valores iniciais da instância Calendario e a meta base de leite (metaLeite) para 100.0f ou 120.0f.

3.2 Interface da Gameplay (HUD e Renderização)

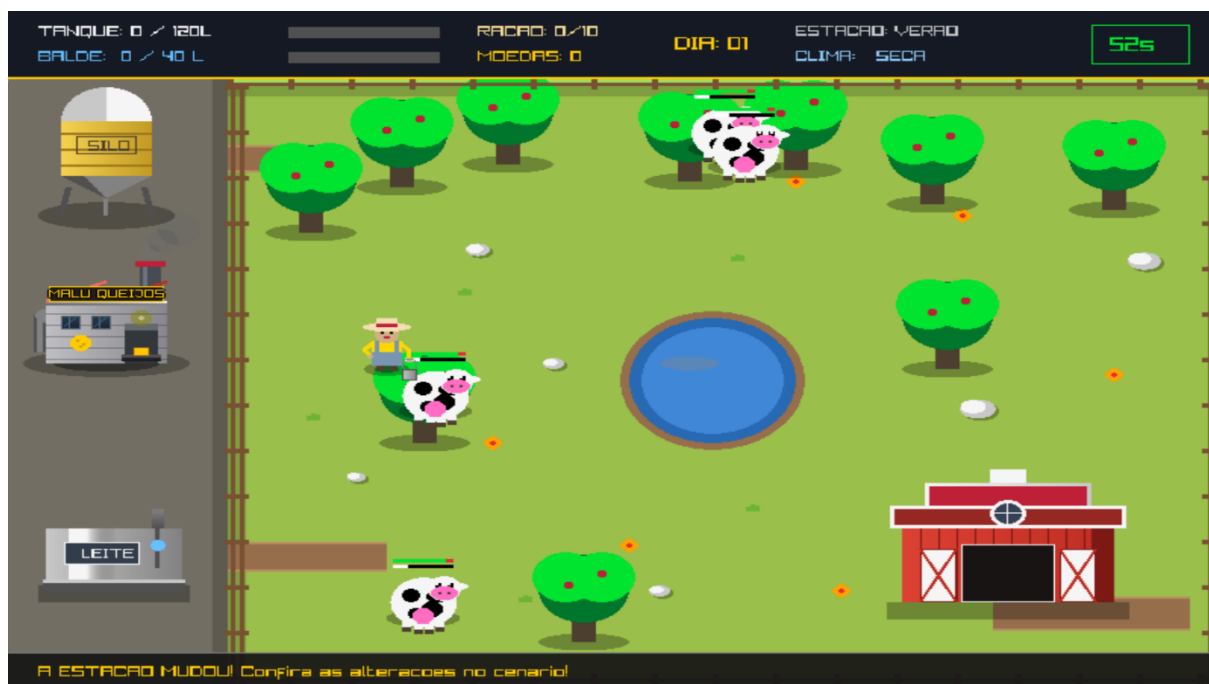


Figura 3 – Demonstração do estado de jogo JOGANDO. A tela exibe no canto superior a HUD informativa atualizada em tempo real. Demonstração da estação VERÃO

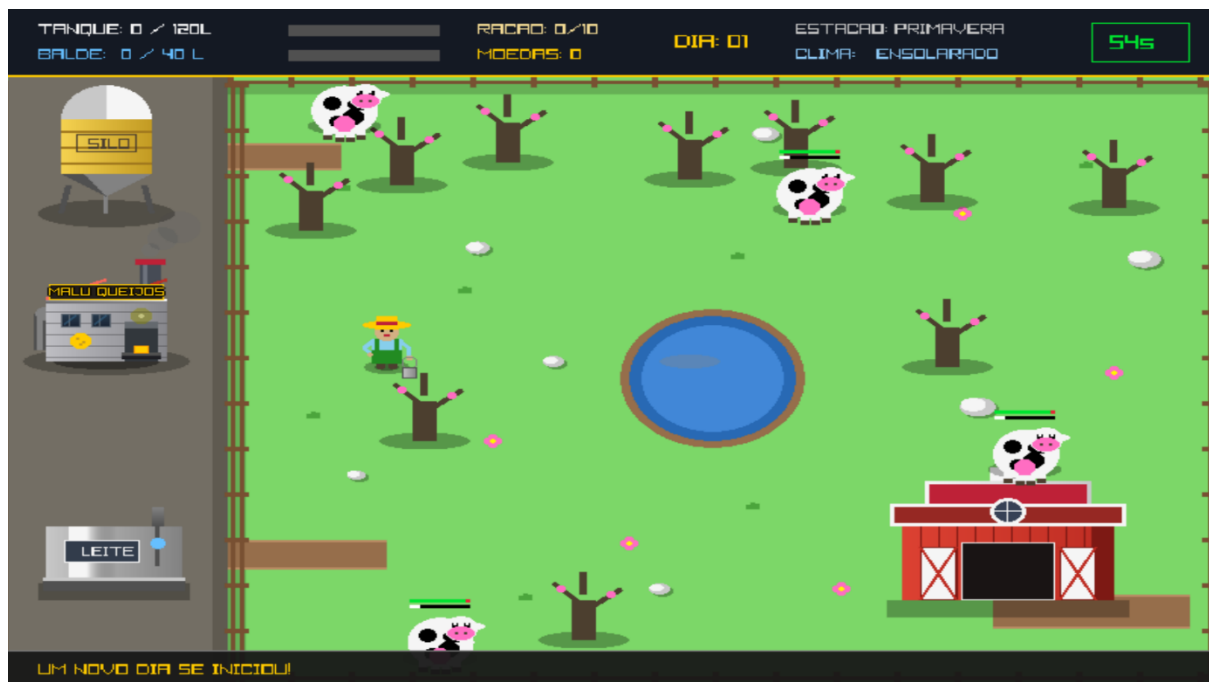


Figura 4 – Demonstração da estação PRIMAVERA



Figura 5 – Demonstração da estação OUTONO

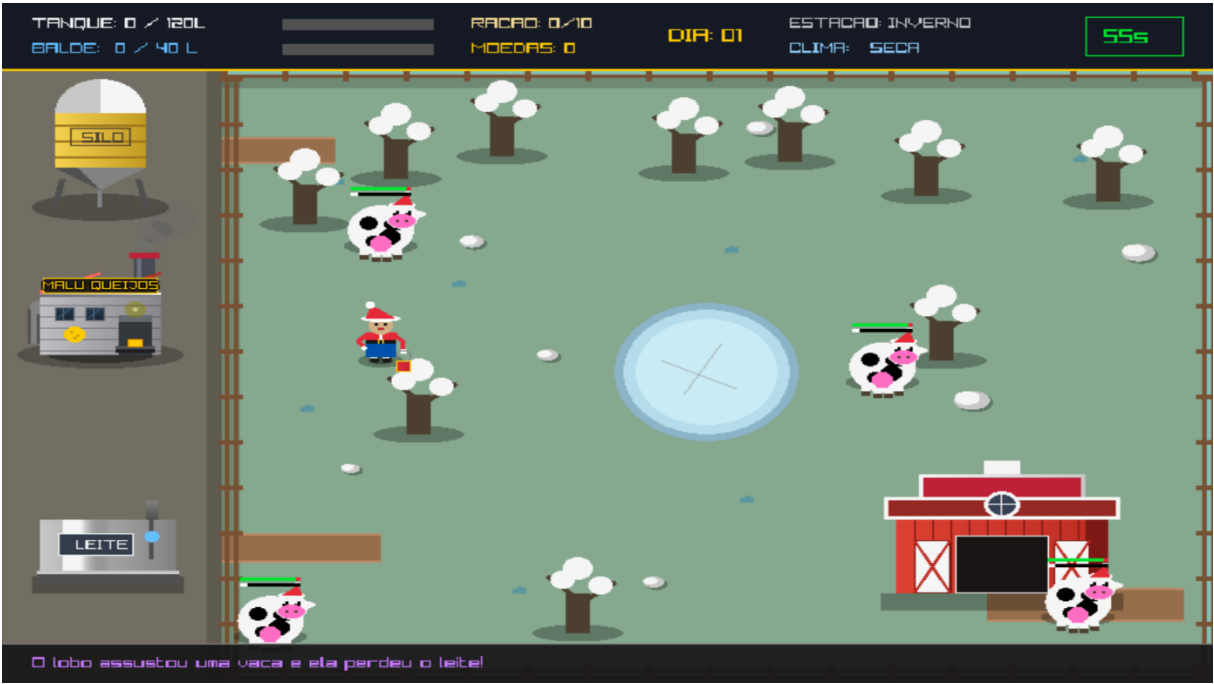


Figura 6 – Demonstração da estação INVERNO

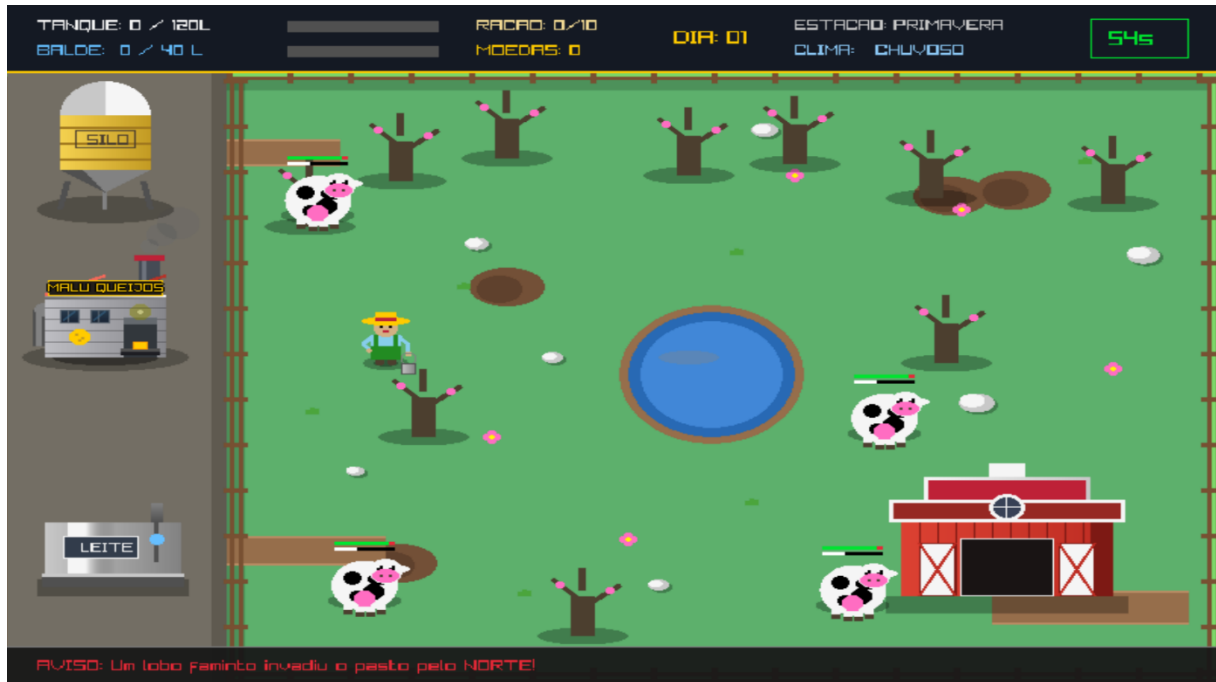


Figura 7 – Demonstração do clima CHUVOSO. É visível o sistema de clima dinâmico aplicando um vetor de poças de lama aleatórias que afetam a velocidade da entidade Jogador por meio do teste geométrico `CheckCollisionCircleRec`.

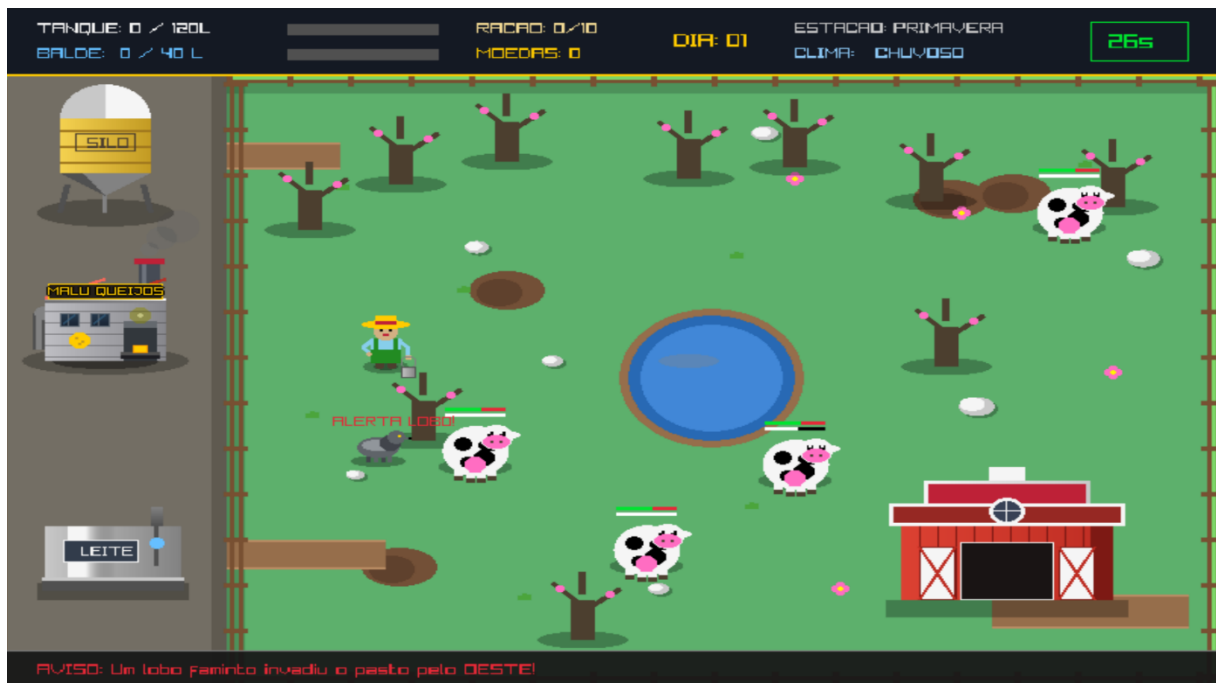


Figura 8 – Demonstração do estado JOGANDO com um ataque de LOBO



Figura 9 – Demonstração do estado JOGANDO com um ataque de LOBO, porém com a mudança da pelugem de acordo com a estação INVERNO

3.3 Interface de seleção de Personagens



Figura 10 – Demonstração da tela de seleção de personagens, o usuário faz a escolha através das teclas [1] e [2].

3.4 Interface de Tutorial



Figura 11 – Demonstração da tela de tutorial (Como Jogar).

3.5 Interface de Game Over e Estatísticas Gerais



Figura 12: Tela de encerramento do jogo (GAME_OVER). A variável `leiteTotalEntregue` monitora a meta do dia corrente e é redefinida a cada manhã, enquanto a métrica persistente `leiteAcumuladoGeneral` exibe com precisão ao jogador o histórico de toda a sua produção acumulada ao longo da partida.

4 CONCLUSÃO

O desenvolvimento de *CowGame: Nolan's Farm* cumpriu os objetivos pedagógicos e técnicos planejados. A adoção da linguagem C++ permitiu compreender de forma prática a importância do gerenciamento de memória através do uso adequado de contêineres dinâmicos (`std::vector`) e da correta destruição de texturas e contextos alocados na VRAM da GPU utilizando o método destrutor da classe `Game`.

A separação lógica entre a variável de controle de metas diárias e a variável de acumulação de score global solucionou o problema de perda de dados estatísticos na transição de turnos do calendário.

Como melhorias futuras sugeridas para o projeto, indicam-se:

- **Serialização de Dados:** Implementar rotinas em C++ (`std::ofstream`) para salvar permanentemente o progresso do jogador e a pontuação máxima (*High Score*) em arquivos binários externos ou `.json`.

- **Expansão do Rebanho:** Adicionar herança à classe Vaca para permitir diferentes raças de animais, cada uma com taxas de consumo de ração e produção de leite distintas.
- **Áudio Posicional:** Integrar o módulo *raudio* da Raylib para adicionar efeitos sonoros tridimensionais ao grito do fazendeiro e à aproximação do lobo predador.

REFERÊNCIAS

- ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS – ABNT. **NBR 6023:** informação / referências / elaboração. Rio de Janeiro, 2002.
- STROUSTRUP, Bjarne. **A Linguagem de Programação C++**. 4. ed. Porto Alegre: Bookman, 2013.
- RAYLIB GRAPHICS LIBRARY. **Raylib API Reference and Documentation**. Disponível em: <https://www.raylib.com>.